

Frontend Developer & Setup Guide

Comprehensive Zero-to-Advanced Setup, Installation, Dependencies, API Testing & Deployment Manual

Target Audience: Beginner to Advanced Developers	Stack: React 18, TypeScript, Vite 5, Tailwind CSS
Document Version: v1.0.0 (Production Release)	Date / Status: October 2026 Verified & Active

1. Document Overview & Executive Summary

Welcome to the official developer and administrator setup guide for the **Rohit Jangir Portfolio & Corporate Platform** frontend application. This project is a modern, high-performance, responsive Single Page Application (SPA) engineered with **React 18**, **TypeScript**, **Vite 5**, and **Tailwind CSS**.

The web application showcases the entrepreneurial ventures, investment portfolio, mentorship initiatives, and case studies of **Rohit Jangir**, including companies such as Aronix Web Tech, Aaru Mobility, Aaru Developers, and the Aaru Care Foundation. This manual is formatted as an end-to-end **zero-to-advanced, step-by-step developer guide**. Whether you are setting up the project on a brand-new computer or preparing for a production deployment on cloud infrastructure, every required tool, terminal command, configuration file, and verification procedure is documented below.

KEY INSTRUCTION: Every terminal command in this guide specifies exactly **what it does** and **in which directory to execute it**. Always verify prerequisites before running the project commands.

2. Required Software, Tools & Official Download Guide

To build and run the frontend application, the following tools must be installed on your development machine. Follow the search instructions below to download the official installers from Google or your search engine of choice:

Software / Tool	Recommended Version	Google Search Query	Official Download URL & Purpose
Node.js & npm	v18.x or v20.x LTS (v20+ recommended)	download nodejs lts	nodejs.org/en/download JavaScript runtime & Node Package Manager to run Vite & install packages.
Git SCM	v2.40+ or latest	download git scm	git-scm.com/downloads Source control management tool to clone and version the codebase.
Visual Studio Code	Latest Stable	download vs code	code.visualstudio.com Recommended Integrated Development Environment (IDE) with TypeScript support.

Web Browser	Chrome / Firefox / Edge	download google chrome	google.com/chrome Modern browser equipped with DevTools for inspecting mobile viewports and network calls.
Postman (Optional)	Latest Desktop App	download postman	postman.com/downloads API testing tool to verify backend contact and pitch endpoints independently.

Recommended VS Code Extensions:

- **Tailwind CSS IntelliSense** (by Tailwind Labs): Provides autocomplete, syntax highlighting, and class linting.
- **ESLint** (by Microsoft): Displays code quality warnings and catches React hook bugs in real time.
- **Prettier - Code formatter**: Ensures consistent formatting across all TypeScript and CSS files.
- **PostCSS Language Support**: Syntax highlighting for custom CSS directives such as @tailwind.

3. Step-by-Step Installation for macOS, Windows & Linux

A. macOS Installation:

1. Download the macOS installer (.pkg) from **nodejs.org** or install via Homebrew.
2. Double-click the .pkg file and follow the on-screen prompts to complete installation.
3. Open **Terminal** (press Cmd + Space, type Terminal, press Enter) and verify:

```
node -v
npm -v
git --version
```

B. Windows Installation:

1. Download the Windows Installer (.msi) 64-bit from **nodejs.org**.
2. Run the installer. On the 'Custom Setup' screen, ensure '**Add to PATH**' is checked.
3. Download Git from **git-scm.com** and install using default recommended settings.
4. Open **PowerShell** or **Command Prompt** (Win + R, type powershell) and verify:

```
node -v
npm -v
git --version
```

C. Linux (Ubuntu / Debian) Installation:

Run the following commands in your bash terminal to install Node.js 20 LTS via NodeSource:

```
# Update package indices
sudo apt update && sudo apt install -y curl git

# Add NodeSource repository for Node.js 20 LTS
```

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
  
# Install Node.js & npm  
sudo apt install -y nodejs  
  
# Verify installation  
node -v && npm -v && git --version
```

4. Project Download / Clone & Terminal Navigation

Follow these exact steps to clone the project from GitHub/GitLab to your local computer:

Step 1: Choose or Create Your Working Directory

Open your terminal and navigate to where you store projects (e.g. Desktop or Projects):

```
# Navigate to your Desktop folder  
cd ~/Desktop  
  
# Or create a dedicated development directory  
mkdir -p ~/Projects && cd ~/Projects
```

Step 2: Clone the Repository

Run the git clone command with the repository URL:

```
# Clone the repository to your machine  
git clone https://github.com/Aronixwebtech001/Rohit_Portfolio_FE.git  
  
# Enter the cloned project folder  
cd Rohit_Portfolio_FE
```

Step 3: Open in Visual Studio Code

Launch VS Code directly inside the project root folder:

```
code .
```

Explanation: The `code .` command tells VS Code to open the current working directory as your workspace.

5. Project Architecture & Complete Folder Structure

The project follows a clean, modular React component architecture structured around domain-specific pages and reusable shared components:

5.1 Root Configuration & Build Manifests:

```
Rohit_Portfolio_FE/  
■■■ index.html           # HTML entry point (SEO meta, fonts, viewport config)  
■■■ package.json         # Project dependencies, scripts, and package metadata  
■■■ package-lock.json    # Deterministic dependency lock file  
■■■ vite.config.ts       # Vite configuration and React plugin setup  
■■■ tailwind.config.js   # Tailwind CSS design system tokens (colors, fonts, key  
frames)  
■■■ postcss.config.js    # PostCSS configuration with Tailwind and Autoprefixer  
■■■ tsconfig.json        # Master TypeScript project configuration  
■■■ tsconfig.app.json    # Application-level TypeScript strict compiler options  
■■■ public/              # Static unbundled public assets (favicon, icons.svg)  
■■■ src/                 # React source code root (detailed below)
```

5.2 Source Code (src/) Architecture:

```

src/
  ■■■ main.tsx                # React root mount point (renders App in #root)
  ■■■ App.tsx                 # React Router configuration & route declarations
  ■■■ index.css               # Global CSS, Tailwind directives, reveal animations &
  touch rules
  ■■■ assets/                 # Bundled assets (branded images, portraits, responsive
  cutouts)
  ■■■ components/
    ■ ■■■ Home/               # HeroSection, PartnerLogos, WhyChooseUs, Partnerships,
    CaseStudies
    ■ ■■■ About/              # ProfileHero, BioSection, MediaImpactSection, QuoteBan
    ner, Journey
    ■ ■■■ Ventures/           # VenturesHero, VenturesDiagram, PortfolioStats, Missio
    nVision
    ■ ■■■ Investor/           # InvestorHero, InvestmentThesis, HowToInvest, Investor
    Form
    ■ ■■■ Mentorship/         # MentorshipHero, TopicsSection, MentorshipCalendar, Pr
    icing
    ■ ■■■ Pitch/              # PitchHero, WhatILookFor, PitchForm
    ■ ■■■ CaseStudy/          # CaseStudyHero, StatsSection, ClientLogosGrid, FAQSect
    ion
    ■ ■■■ Resources/          # ResourcesHero, ArticlesGrid
    ■ ■■■ shared/             # Navbar, Footer, Layout, ScrollReveal, ConnectModal, B
    utton
  ■■■ features/               # Feature API client modules:
    ■ ■■■ investor/           # api.investor.ts (POST /investors), types.investor.ts
    ■ ■■■ pitch/              # api.pitch.ts (POST /pitch/submit), types.pitch.ts
    ■ ■■■ subscribe/          # api.subscribe.ts (POST /subscribe), types.subscribe.t
    s
  ■■■ lib/
    ■ ■■■ api.ts              # Generic fetch wrapper with baseUrl & error parsing
  ■■■ pages/                  # Top-level route components mapped in App.tsx
  ■■■ types/                  # Global TypeScript type definitions

```

6. Configuration & Environment Variables (.env)

Vite utilizes standard .env files for environment variables. **Important Vite Rule:** To prevent accidentally leaking private credentials to the client browser, only variables prefixed with VITE_ are exposed to your React code via import.meta.env.

Step 1: Create Your Local .env File

In the project root directory (Rohit_Portfolio_FE), create a new .env file:

```
# macOS / Linux
cp .env.example .env 2>/dev/null || touch .env

# Windows (PowerShell)
if (!(Test-Path .env)) { New-Item -ItemType File -Name .env }
```

Step 2: Add Configuration Keys

Open .env in your code editor and populate the required keys:

```
# API Backend Base URL (FastAPI / Express / Node.js backend)
VITE_API_URL=http://localhost:5000/api/v1

# Application Environment Flag
VITE_APP_ENV=development

# Cloudinary Assets CDN URL (Already configured for streaming video & images)
VITE_CLOUDINARY_BASE=https://res.cloudinary.com/dqfuozgjq
```

Variable Key	Default / Development Value	Purpose & Where Used
VITE_API_URL	http://localhost:5000/api/v1	Backend API root. Read in src/lib/api.ts to prefix all REST endpoints.
VITE_APP_ENV	development	Toggles debug logs and dev tooling between dev and production modes.
VITE_CLOUDINARY_BASE	https://res.cloudinary.com/dqfuozgjq	Cloudinary CDN root hosting videos and images with q_auto,w_900 compression.

7. Dependencies & Packages Installation

All project dependencies are declared in package.json. Run the following command in the project root directory to download and install all libraries into node_modules/:

```
# Execute inside: /Users/.../Desktop/Rohit_Portfolio_FE
npm install
```

What this does: npm install reads package-lock.json to ensure exact matching package versions are resolved and installed into your local node_modules/ folder.

Breakdown of Key Dependencies:

Package Name	Version	Role in Application
react & react-dom	^18.3.1	The core UI library and DOM renderer for component hierarchy, hooks, and Virtual DOM.
react-router-dom	^6.26.2	Client-side SPA router providing declarative navigation without full page refreshes.
lucide-react	^1.33.0	High-performance vector SVG icons (e.g. ChevronDown, Menu, X, ArrowRight).
react-countup	^6.5.3	Smooth animated numerical counters used in venture statistics and impact metrics.
react-intersection-observer	^11.0.1	Tracks element viewport visibility to trigger scroll-reveal animations.
vite	^5.4.2	Next-generation frontend tooling providing lightning-fast HMR and Rollup production builds.
tailwindcss & autoprefixer	^3.4.10	Utility-first styling framework configured with luxury theme tokens and vendor prefixing.
typescript	^5.5.4	Type safety, autocompletion, interface enforcement, and compile-time error detection.

8. How to Run the Project (Development Server)

To start the Vite local development server with Instant Hot Module Replacement (HMR):

```
# Run in project root: Rohit_Portfolio_FE/
npm run dev
```

Once started, Vite will output the local network URL in your terminal:

```
VITE v5.4.2 ready in 240 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Open your browser and navigate to **http://localhost:5173** (or the port indicated in your terminal).

Advanced Development Options:

- **Test on Mobile Phones over Wi-Fi:** Run `npm run dev -- --host`. Vite will output an IP address (e.g. `http://192.168.1.15:5173`) that you can open on your mobile phone or tablet.
- **Specify a Custom Port:** Run `npm run dev -- --port 3000` if port 5173 is already in use by another application.
- **Stop the Dev Server:** Press `Ctrl + C` in your terminal window.

9. Database Setup, API Configuration & Testing

Frontend Database Architecture: As a modern client-side Single Page Application, the frontend does not connect directly to a database (such as MongoDB or PostgreSQL) for security reasons. Instead, the frontend communicates with the backend REST API via secure HTTP endpoints defined in `src/lib/api.ts`. The backend server manages the database connection.

Frontend API Modules & Endpoints:

Feature Form	File Location	HTTP Method & Endpoint	Payload Fields
Investor Form	<code>src/features/investor / api.investor.ts</code>	POST <code>/investors</code>	name, email, phone, ticketSize, investmentFocus, message
Pitch Submission	<code>src/features/pitch/ api.pitch.ts</code>	POST <code>/pitch/submit</code>	founderName, email, startupName, stage, pitchDeckUrl, message
Newsletter Subscription	<code>src/features/subscrib e/ api.subscribe.ts</code>	POST <code>/subscribe</code>	email
Quick Connect Modal	<code>src/components/shared / ConnectModal.tsx</code>	POST <code>/connect</code>	name, email, purpose, message

Testing API Endpoints with cURL Commands:

You can verify backend API connectivity directly from your terminal using these exact commands:

```
# 1. Test Investor Application Submission
curl -X POST "http://localhost:5000/api/v1/investors" \
  -H "Content-Type: application/json" \
  -d '{"name":"Jane Doe","email":"jane@investor.com","ticketSize":"$100K-$250K","message":"Interested in mobility portfolio."}'

# 2. Test Pitch Submission
curl -X POST "http://localhost:5000/api/v1/pitch/submit" \
  -H "Content-Type: application/json" \
  -d '{"founderName":"John Smith","email":"john@startup.com","startupName":"NextGen AI","stage":"Seed"}'

# 3. Test Newsletter Subscription
curl -X POST "http://localhost:5000/api/v1/subscribe" \
  -H "Content-Type: application/json" \
  -d '{"email":"subscriber@example.com"}'
```

10. Build & Production Setup

Before deploying to production, compile the application into static, minified production assets:

```
# Run in project root: Rohit_Portfolio_FE/  
npm run build
```

What happens during npm run build:

1. **TypeScript Type Checking:** Executes `tsc -b`. Checks every component and feature for strict type compliance.
2. **Vite Rollup Bundling:** Packages all TSX, CSS, and media imports into high-performance chunks.
3. **Asset Optimization:** Generates content-hashed filenames for immutable browser caching (e.g. `index-CbMISFpc.js`).
4. **Output Generation:** Writes all compiled files into the standalone `dist/` folder.

Previewing the Production Build Locally:

Verify that the compiled build works exactly as expected before deploying:

```
# Starts a local static web server serving the dist/ folder  
npm run preview
```

Open **`http://localhost:4173`** to test the production bundle in your browser.

11. Production Deployment Instructions

The compiled `dist/` directory can be deployed to any modern cloud platform or traditional web server. Because this application uses **React Router** client-side routing, your server must rewrite all page requests to `/index.html`.

A. Deploy to Vercel (Recommended):

1. Push your repository to GitHub or GitLab.
2. Log into **vercel.com**, click '**Add New Project**', and select your repository.
3. Configure Build Settings: Framework Preset = **Vite**, Build Command = `npm run build`, Output Directory = `dist`.
4. Add Environment Variable: `VITE_API_URL` with your live backend production URL.
5. Click **Deploy**. Vercel automatically configures SPA routing and SSL.

B. Deploy to Netlify:

Create a file named `public/_redirects` containing the following single rule:

```
/*      /index.html    200
```

Connect your Git repo on **netlify.com** with build command `npm run build` and publish directory `dist`.

C. Deploy to Ubuntu / Nginx Web Server:

Upload the contents of `dist/` to `/var/www/rohit-portfolio/` and use this Nginx configuration:

```
server {  
    listen 80;  
  
    server_name rohitjangir.com www.rohitjangir.com;  
  
    root /var/www/rohit-portfolio;  
  
    index index.html;  
  
    # Enable gzip compression for lightning-fast loads  
  
    gzip on;  
  
    gzip_types text/plain text/css application/json application/javascript text/xml image  
/svg+xml;  
  
    # Essential SPA Rewrite Rule  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    # Cache static assets for 1 year  
  
    location ~* \.(?:ico|css|js|gif|jpe?g|png|woff2?|eot|ttf|svg|mp4)$ {  
        expires 1y;  
    }  
}
```

```
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
}
```

12. Common Errors & Troubleshooting Solutions

Error / Symptom	Root Cause	Step-by-Step Solution
Port 5173 is in use	Another terminal instance of Vite or Node is already running on port 5173.	Vite automatically falls back to 5174. To free 5173: run <code>lsof -i :5173</code> followed by <code>kill -9 <PID></code> .
VITE_API_URL is undefined	Missing .env file or Vite dev server was not restarted after editing .env.	Create .env in the project root, ensure the variable starts with VITE_, and restart Vite (<code>npm run dev</code>).
Cannot find module or type declaration	Packages have not been installed into node_modules/.	Run <code>npm install</code> in the project root. If persistent, delete node_modules and reinstall.
CORS Policy: No Access-Control header	The backend server does not allow cross-origin requests from the frontend port.	In backend (FastAPI/Express), add <code>http://localhost:5173</code> to <code>CORSMiddleware allow_origins</code> .
404 Not Found on page refresh	Web server (Nginx/Netlify) is treating client route (e.g. /about) as a physical directory.	Add fallback rewrite rule: <code>try_files \$uri \$uri/ /index.html;</code> (Nginx) or <code>/* /index.html 200</code> (Netlify).
Horizontal scrollbar / screen jitter on mobile	Elements with <code>translateX</code> animation or fixed widths wider than viewport.	Already resolved! <code>index.css</code> enforces <code>overflow-x: clip</code> and <code>maps horizontal reveals to translateY</code> on $\leq 768\text{px}$.

13. Comprehensive Troubleshooting & Clean Reinstall

If you encounter anomalous behavior, corrupted dependencies, or stale build caches, perform a complete clean reinstall using these commands:

```
# 1. Stop any running Vite processes (Ctrl + C)

# 2. Remove dependencies and build cache
rm -rf node_modules package-lock.json dist .vite

# 3. Clear npm global cache
npm cache clean --force

# 4. Reinstall all dependencies fresh
npm install

# 5. Test clean build
npm run build

# 6. Launch development server
npm run dev
```

14. Final System Verification Checklist

Use this quality assurance checklist to verify that your development and production setup is 100% operational:

Verification Item	Inspection Action	Expected Status
1. Node & npm Version	Run node -v && npm -v in terminal.	Node ≥ v18.0.0, npm ≥ 9.0.0
2. Development Server	Run npm run dev and open http://localhost:5173.	Instant load, no terminal errors, HMR active
3. Navigation & Routing	Click Home, About, Ventures, Investor, Mentorship, Pitch links.	Smooth client-side routing, URL updates cleanly
4. Video Playback	Inspect 'Why we're the right choice' video on Home page.	Autoplays muted, sharp poster, 1.9MB payload
5. Case Studies Carousel	Test on phone screen (< 640px) and tablet (< 1024px).	Cards full width, bottom arrows & dots, swipe active
6. Partnerships Section	Inspect top padding of 'Partnerships' white card.	48px top space, heading has elegant breathing room
7. Quote Banner (About)	Inspect About page quote banner on mobile.	Rohit's portrait centered in background, no text-justify

8. Zero Horizontal Scroll	Scroll horizontally on mobile viewports (375px & 414px).	Page strictly vertical, zero horizontal overflow
9. Production Build	Run npm run build in terminal.	Exits with code 0 in < 5s, creates dist/ folder

15. Developer Maintenance & Extension Guidelines

Adding a New Page / Route:

1. Create a new component in src/pages/NewPage.tsx.
2. In src/App.tsx, import the page and declare a new route inside <Routes>:
 <Route path="/new-page" element={<NewPage />} />.
3. Add a navigation link in src/components/shared/Navbar.tsx and Footer.tsx.

Modifying Color Tokens & Theme:

All brand colors are centralized in tailwind.config.js under theme.extend.colors. Updating values for navy, accent, cream, or card propagates consistently across every component in the project.

DOCUMENTATION METADATA & SUPPORT: Documentation Maintained by: Aronix Web Tech Engineering Team

Official Website: <https://aronixwebtech.com/>

Project Repository: https://github.com/Aronixwebtech001/Rohit_Portfolio_FE

Inquiries & Support: contact@aronixwebtech.com